



Universidad Autónoma de Chihuahua
Facultad de Ingeniería



Título

Proyecto Final

Aplicación Spring Boot

Materia

Desarrollo Basado en Plataformas

Docente

Oscar Beltran Gómez

Equipo “Uno” - Integrantes

377043 – Angel Rodriguez Palomino

377304 – Zahid Alberto Jimenes Pérez

377061 – Jared Isai López Espino

Fecha de entrega

30 de mayo de 2026

Requisitos para presentar el proyecto final

1. Repositorio en GitHub
2. Implementación sobre el framework Spring Boot
3. Manejo de base de datos (altas, bajas, cambios y consultas)
4. Implementación de al menos 2 rutas de Apache Camel
5. Plantilla de estilo admin
6. Conexión con RabbitMQ
7. Opcional conexión a REDIS

Sistema/Aplicación

Nombre del Sistema:

Sistema de Carnet Cultural UACH

Descripción del Sistema

El Sistema de Carnet Cultural es una aplicación web diseñada para gestionar la participación de estudiantes en eventos universitarios culturales, científicos, deportivos y de apoyo comunitario.

El sistema permite que los empleados registren y administren eventos, mientras que los estudiantes pueden consultar actividades disponibles, registrarse y subir evidencias de asistencia.

Objetivo General

Desarrollar una aplicación web para la administración del carnet cultural universitario, permitiendo el registro de eventos, control de asistencia de estudiantes y validación de evidencias.

Objetivos específicos

Centralizar la información de estudiantes y eventos culturales, facilitar el registro de asistencias, permitir la consulta del avance del carnet cultural, generar reportes de participación cultural y mejorar la seguridad y control de acceso a la información.

Permite mejorar la experiencia del usuario al mandar evidencias en un lugar centralizado en vez de mandar correos como el sistema actual.

Descripción de tipos de usuarios

El sistema está hecho para abarcar dos tipos de usuarios, empleados y usuarios.

Empleado

Usuario encargado de administrar el sistema.

Funciones:

- Registrar eventos.
- Validar asistencias.
- Administrar eventos.
- Generar formularios.

Estudiante

Usuario que participa en actividades culturales y tiene acceso al sistema.

Funciones:

- Consultar eventos.
- Ver historial de asistencias.
- Consultar créditos acumulados.
- Consultar avance del carnet cultural.

Requerimientos funcionales

Centrándose en lo que es el Carnet Cultural...

- El sistema debe permitir iniciar sesión, tanto del estudiante y empleado (usuarios)
- El sistema debe permitir registrar eventos, sin importar la categoría, permite ingresar descripciones breves de ello y su puntaje.

- El sistema debe desplegar el historial de eventos del usuario, un usuario podrá ver que registros de eventos ya tiene registrados para su Carnet.
- El sistema debe permitir administrar eventos (crear/agendar), un empleado puede generar una entrada de evento para que los estudiantes puedan acceder al registro.
- El sistema debe permitir aceptar o rechazar registros de evidencias, un empleado se le permite observar la evidencia del registro de los usuarios, aceptarlo significa guardarlo en la base de datos, rechazarlo significa anularlo del registro del usuario.

Arquitectura del Sistema/Aplicación

Se sigue la arquitectura de 3 capas, las cuales son:

- Capa de presentación, aquí se encuentra la interfaz gráfica utilizada por estudiantes y empleados en el sistema siendo la pestaña de login y la pestaña de dashboard correspondiente.
- Capa lógica, aquí se procesa todo sobre el sistema, validaciones y el manejo de las solicitudes de login.
- Capa de datos, aquí es donde se controla o se gestiona la base de datos, que la persistencia e integridad de los datos se cumpla.

Tecnologías de Desarrollo

Tecnologías Backend

- Spring Boot
- Spring Data JPA
- Oracle Database
- Redis
- Apache Camel
- Rabbitmq

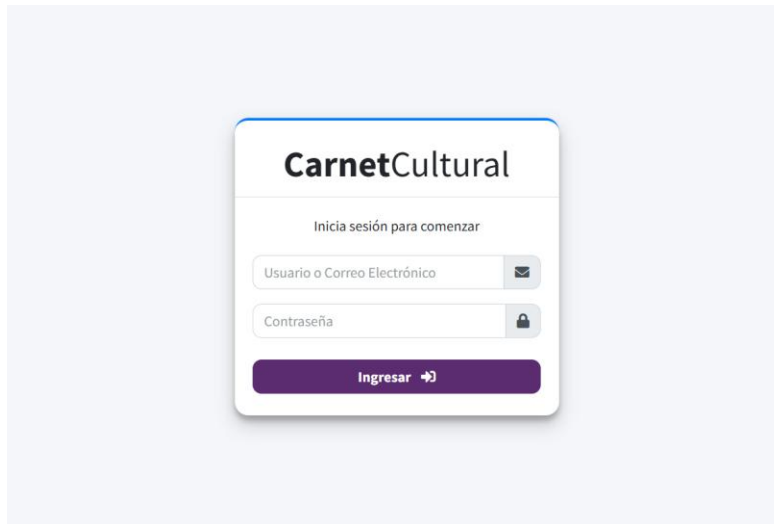
- Tecnologías Frontend

- HTML5
- CSS

- JavaScript
- Thymeleaf

Interfaz de la aplicación

Inicio de sesión



The login form is a white card with rounded corners and a blue border, centered on a light blue background. It features the 'CarnetCultural' logo at the top, followed by the instruction 'Inicia sesión para comenzar'. Below this are two input fields: 'Usuario o Correo Electrónico' with an email icon and 'Contraseña' with a lock icon. A purple 'Ingresar' button with a right arrow is at the bottom.

CarnetCultural

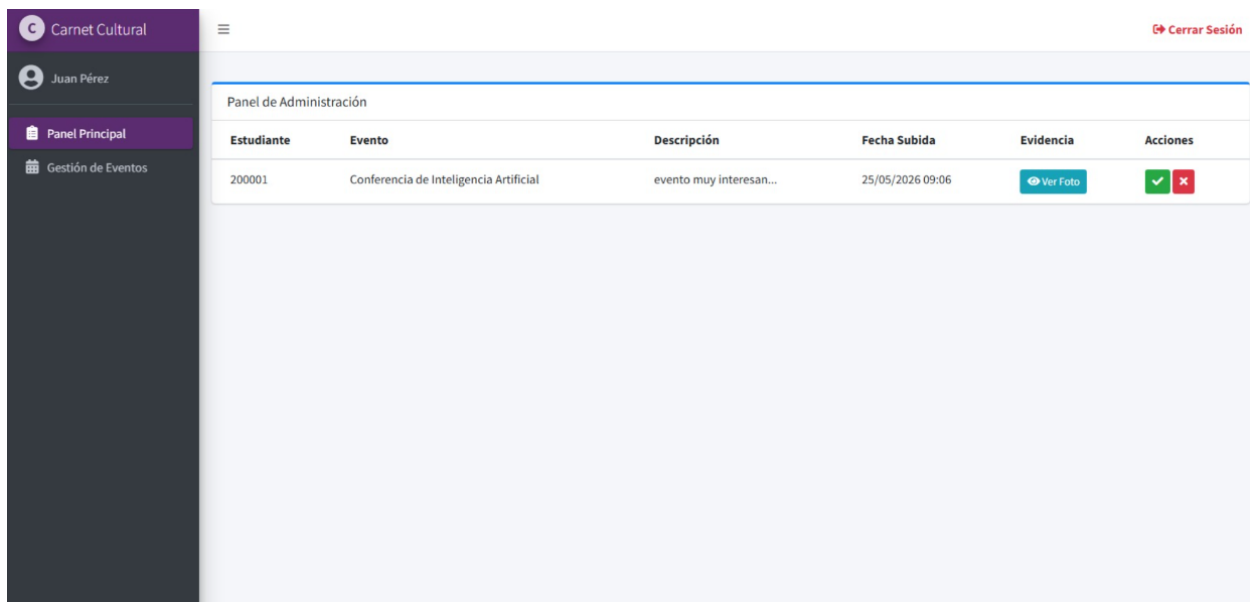
Inicia sesión para comenzar

Usuario o Correo Electrónico

Contraseña

Ingresar ➔

Panel de empleado



The dashboard has a dark purple sidebar on the left with the 'Carnet Cultural' logo, the user's name 'Juan Pérez', and two menu items: 'Panel Principal' (active) and 'Gestión de Eventos'. The main content area has a top bar with a 'Cerrar Sesión' link. Below is a 'Panel de Administración' section containing a table with event data.

Carnet Cultural

Juan Pérez

Panel Principal

Gestión de Eventos

Cerrar Sesión

Panel de Administración

Estudiante	Evento	Descripción	Fecha Subida	Evidencia	Acciones
200001	Conferencia de Inteligencia Artificial	evento muy interesan...	25/05/2026 09:06	Ver Foto	✓ ✗

Gestión de eventos

Carnet Cultural

Juan Pérez

Panel Principal

Gestión de Eventos

Cerrar Sesión

Gestión de Eventos

+ Crear Nuevo Evento

Nombre	Fecha Evento	Fecha Fin	Ubicación	Puntos	Categoría
evento generico1	17/06/2026 11:59	-----	fing	2	Artístico
evento generico2	02/07/2026 00:00	02/07/2026 14:30	mi casa	3	Salud
Festival de Música Universitaria	15/06/2026 00:00	15/06/2026 14:00	Auditorio Universitario	1	Artístico
Torneo Interfacultades	20/06/2026 00:00	20/06/2026 18:00	Gimnasio Universitario	2	Deportivo
Conferencia de Inteligencia Artificial	01/07/2026 00:00	01/07/2026 12:00	Sala de Conferencias	3	Científico-Filosofico
Campaña de Reforestación	10/07/2026 00:00	10/07/2026 16:00	Parque Metropolitano	1	Comunitario

Consultar próximos eventos

Carnet Cultural

Carlos Ramírez

Eventos Cumplidos

Subir Evidencia

Próximos Eventos

Cerrar Sesión

Próximos Eventos

Cartelera de Actividades

Nombre	Tipo	Fecha Evento	Fecha Fin	Ubicación	Puntos
evento generico2	Salud	02-Jul-2026 00:00	02-Jul-2026 14:30	mi casa	3
Festival de Música Universitaria	Artístico	15-Jun-2026 00:00	15-Jun-2026 14:00	Auditorio Universitario	1
Torneo Interfacultades	Deportivo	20-Jun-2026 00:00	20-Jun-2026 18:00	Gimnasio Universitario	2
Conferencia de Inteligencia Artificial	Científico-Filosofico	01-Jul-2026 00:00	01-Jul-2026 12:00	Sala de Conferencias	3
Campaña de Reforestación	Comunitario	10-Jul-2026 00:00	10-Jul-2026 16:00	Parque Metropolitano	1
Actividades Permanentes					
evento generico1	Artístico	17-Jun-2026	-----	fing	2

Interfaz de estudiante

Carnet Cultural

Carlos Ramírez

Eventos Cumplidos

Subir Evidencia

Próximos Eventos

Cerrar Sesión

Mis Eventos Cumplidos

1/6 Artística

0/6 Científico-Filosofica

2/6 Deportiva

0/8 Herramientas para mi Futuro

0/6 Salud

1/6 Apoyo a tu Comunidad

Historial de Eventos Aprobados

Evento	Categoría	Fecha Confirmación	Valor
Torneo Interfacultades	Deportivo	25/05/2026	+2 Puntos
Festival de Música Universitaria	Artístico	25/05/2026	+1 Puntos
Campaña de Reforestación	Comunitario	25/05/2026	+1 Puntos

Subir evidencias

The screenshot shows a web application interface for uploading event evidence. On the left is a dark sidebar with the 'Carnet Cultural' logo and user 'Carlos Ramírez'. The main area is titled 'Subir Evidencia de Evento'. It contains a form with three sections: 'Detalles de la Evidencia', 'Nombre del Evento' (a dropdown menu currently showing 'Conferencia de Inteligencia Artificial'), and 'Descripción del evento' (a text area with 'evento muy interesante' and a 'Máximo 1000 caracteres' limit). Below this is the 'Evidencia (Solo imágenes)' section, which shows a file named 'ia.jfif' with a 'Browse' button and a red 'X' icon. A link 'Ver vista previa de: ia.jfif' is also present. At the bottom right of the form is a purple 'Enviar Evidencia' button. A red 'Cerrar Sesión' link is in the top right corner.

La interfaz de la aplicación utiliza la plantilla admin trabajada a lo largo del curso.

Para el uso de la base de datos, nosotros usamos la base de datos de Oracle, la cual requiere del uso de dependencias en Spring Boot como JPA y el driver de Oracle, ahora para hacer uso de cada tabla de nuestra base de datos nosotros creamos un script el cual contiene tanto la estructura de cada tabla, constraints y datos de prueba.

```
▼ repositories
  J CategoriaRepository.java
  J EmpleadoRepository.java
  J EstudianteRepository.java
  J EventoRepository.java
  J FacultadRepository.java
  J RegistroAsistenciaRepository....
  J UsuarioRepository.java
```

Se utiliza los repositories para poder implementar operaciones CRUD para cada tabla, si es necesario también es posible agregar operaciones más específicas.

Así se ve EventoRepository y un ejemplo Read (búsqueda)

```
package com.carnet.uach.repositories;

import com.carnet.uach.models.Evento;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;
import java.util.List;

import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.query.Param;

@Repository
public interface EventoRepository extends JpaRepository<Evento, Long> {

    @Query("SELECT e FROM Evento e WHERE e.fechaFin IS NULL OR e.fechaFin > :fecha")
    List<Evento> findDisponibles(@Param("fecha") java.time.LocalDateTime fecha);
}
```

```
/**
 * Muestra el formulario para crear un nuevo evento.
 */
@GetMapping("/nuevo")
public String mostrarFormularioNuevo(Model model) {
    model.addAttribute("evento", new Evento());
    model.addAttribute("categorias", categoriaRepository.findAll());
    return "empleado/evento-form";
}
```

Rabbitmq

Para poder hacer uso del servicio de rabbitmq, primero establecemos la conexión:

```
# =====
# Configuración RabbitMQ
# =====
spring.rabbitmq.host=localhost
spring.rabbitmq.port=5672
spring.rabbitmq.username=guest
spring.rabbitmq.password=guest
```

Lo usaremos para crear una cola y que se encargue de direccionar los mensajes utilizando también de apoyo Apache camel.

```

package com.carnet.uach.config;

import org.springframework.amqp.core.Binding;
import org.springframework.amqp.core.BindingBuilder;
import org.springframework.amqp.core.DirectExchange;
import org.springframework.amqp.core.Queue;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
public class RabbitMQConfig {

    public static final String QUEUE_EVIDENCIAS = "evidencias.rechazadas.queue";
    public static final String EXCHANGE_EVIDENCIAS = "evidencias.exchange";
    public static final String ROUTING_KEY_EVIDENCIAS = "evidencia.rechazada";

    @Bean
    public Queue evidenciasQueue() {
        return new Queue(QUEUE_EVIDENCIAS, true); // durable
    }

    @Bean
    public DirectExchange evidenciasExchange() {
        return new DirectExchange(EXCHANGE_EVIDENCIAS);
    }

    @Bean
    public Binding bindingEvidencias(Queue evidenciasQueue, DirectExchange evidenciasExchange) {
        return BindingBuilder.bind(evidenciasQueue).to(evidenciasExchange).with(ROUTING_KEY_EVIDENCIAS);
    }
}

```

RabbitMQConfig nos genera todo lo necesario, la cola, el Exchange y la ruta, lo usaremos para poder eliminar la evidencias físicas en caso de que el registro del estudiante fuera rechazado.

RabbitMQ TM RabbitMQ 4.3.0 Erlang 27.3.4.11

Overview Connections Channels **Exchanges** Queues and Streams Admin

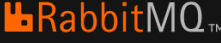
Exchanges

▼ All exchanges (10)

Pagination

Page **1** of 1 - Filter: ☐ Regex ?

Virtual host	Name	Type	Features	Message rate in	Message rate out	+/-
/	(AMQP default)	direct	D			
/	amq.direct	direct	D			
/	amq.fanout	fanout	D			
/	amq.headers	headers	D			
/	amq.match	headers	D			
/	amq.rabbitmq.log	topic	D I			
/	amq.rabbitmq.trace	topic	D I			
/	amq.topic	topic	D			
/	evidencias.exchange	direct	D			


RabbitMQ 4.3.0 Erlang 27.3.4.11

Overview
Connections
Channels
Exchanges
Queues and Streams
Admin

Queues

▼ All queues (2)

Pagination

 Page 1 of 1 - Filter: ☐ Regex ?

Overview					Messages			Message rates				+/-
Virtual host	Name	Type	Features	State	Ready	Unacked	Total	incoming	deliver / get	ack		
/	evidencias.rechazadas.queue	classic	D Args	running	0	0	0					

Apache Camel

Tenemos aparte de la mensajeria de RabbitMQ, usamos apache camel, la primera ruta es para borrar evidencia física o donde se este guardado

```
package com.carnet.uach.routes;

import org.apache.camel.builder.RouteBuilder;
import org.springframework.stereotype.Component;

import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.Paths;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

@Component
public class EvidenciaRechazadaRoute extends RouteBuilder {

    private static final Logger logger = LoggerFactory.getLogger(EvidenciaRechazadaRoute.class);

    @Override
    public void configure() throws Exception {
        // =====
        // INTEGRACIÓN CAMEL + RABBITMQ (CONSUMIDOR)
        // =====
        // Se utiliza el componente 'spring-rabbitmq' de Apache Camel para consumir mensajes
        // de forma asincrónica desde un exchange de RabbitMQ.
        //
        // Parámetros de la URI:
        // - exchangeName: 'evidencias.exchange' (El exchange al que nos conectamos).
        // - queues: 'evidencias.rechazadas.queue' (La cola de donde Camel leerá los mensajes).
        // - routingKey: 'evidencia.rechazada' (La clave de enrutamiento para filtrar mensajes).
        // - autoDeclare=false: Indica que las colas/exchanges ya están declaradas en la configuración.
        // =====
        from("spring-rabbitmq:evidencias.exchange?queues=evidencias.rechazadas.queue&routingKey=evidencia.rechazada&autoDeclare=false")
            .routeId("evidencia-rechazada-route") // Identificador único de la ruta en Camel
            .process(exchange -> {
                // Aquí inicia el procesamiento del mensaje consumido.
                // El cuerpo del mensaje (Body) contiene la ruta del archivo a eliminar,
                // que fue enviado como un String desde el servicio de RegistroAsistencia.
                String rutaArchivo = exchange.getIn().getBody(String.class);
                if (rutaArchivo != null && !rutaArchivo.trim().isEmpty()) {
                    try {
                        Path path = Paths.get(rutaArchivo);
                        // Se realiza la eliminación física del archivo
                        boolean borrado = Files.deleteIfExists(path);
                        if (borrado) {
                            logger.info("Evidencia eliminada físicamente del disco de forma asincrónica: {}", rutaArchivo);
                        } else {
                            logger.warn("El archivo no se encontró en el disco para borrar (posiblemente ya fue eliminado): {}", rutaArchivo);
                        }
                    } catch (Exception e) {
                        logger.error("Error al intentar borrar el archivo físico {}: {}", rutaArchivo, e.getMessage());
                        // Dependiendo de la política de reintentos de Camel/RabbitMQ, se podría relanzar la excepción.
                    }
                }
            });
    }
}
```

```
2026-05-29T16:06:03.364-06:00 INFO 20828 --- [uach] [pool-3-thread-4] c.c.uach.routes.EvidenciaRechazadaRoute : Evidencia eliminada físicamente del disco de forma asíncrona:
ias\eb6af69-8c86-4cee-91a0-b74b6f3d09fe_fing-escudo.png
[]
```

```
2026-05-29T16:07:47.371-06:00 INFO 20828 --- [uach] [ficarEstudiante] c.carnet.uach.routes.NotificacionRoute : =====
2026-05-29T16:07:47.371-06:00 INFO 20828 --- [uach] [ficarEstudiante] c.carnet.uach.routes.NotificacionRoute : ? [NOTIFICACIÓN ENVIADA AL ESTUDIANTE]
2026-05-29T16:07:47.371-06:00 INFO 20828 --- [uach] [ficarEstudiante] c.carnet.uach.routes.NotificacionRoute : Mensaje: Lo sentimos. Tu evidencia para el evento ID: 5 ha sido
2026-05-29T16:07:47.371-06:00 INFO 20828 --- [uach] [ficarEstudiante] c.carnet.uach.routes.NotificacionRoute : =====
2026-05-29T16:07:47.371-06:00 WARN 20828 --- [uach] [pool-3-thread-5] c.c.uach.routes.EvidenciaRechazadaRoute : El archivo no se encontró en el disco para borrar (posiblemente
): uploads/evidencias/4161cbaf-2d02-4fde-86ca-c8f6ec125e0a_escudo-color.png
[]
```

La segunda ruta será para enviar correos a los estudiantes sobre notificaciones, siendo una simulación

```
package com.carnet.uach.routes;

import org.apache.camel.builder.RouteBuilder;
import org.springframework.stereotype.Component;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

@Component
public class NotificacionRoute extends RouteBuilder {

    private static final Logger logger = LoggerFactory.getLogger(NotificacionRoute.class);

    @Override
    public void configure() throws Exception {
        // =====
        // RUTAS DE SIMULACIÓN DE NOTIFICACIONES (USANDO SEDA)
        // SEDA es un componente de Camel que crea colas asíncronas en la memoria de la aplicación.
        // Es perfecto para simulaciones rápidas sin depender de RabbitMQ.
        // =====

        // Ruta 1: Notificar al estudiante cuando se acepta/rechaza su evidencia
        from("seda:notificarEstudiante")
            .routeId("notificar-estudiante-route")
            .process(exchange -> {
                String mensaje = exchange.getIn().getBody(String.class);
                // Aquí simularemos el envío de un correo electrónico real usando camel-mail
                logger.info("=====");
                logger.info("📧 [NOTIFICACIÓN ENVIADA AL ESTUDIANTE]");
                logger.info("Mensaje: {}", mensaje);
                logger.info("=====");
            });

        // Ruta 2: Notificar al encargado (empleado) cuando un estudiante sube una evidencia
        from("seda:notificarEncargado")
            .routeId("notificar-encargado-route")
            .process(exchange -> {
                String mensaje = exchange.getIn().getBody(String.class);
                // Aquí simularemos enviar un mensaje al dashboard del empleado o un correo
                logger.info("=====");
                logger.info("🚨 [ALERTA PARA EL ENCARGADO]");
                logger.info("Mensaje: {}", mensaje);
                logger.info("=====");
            });
    }
}
```

```
--- [uach] [ificarEncargado] c.carnet.uach.routes.NotificacionRoute : =====
--- [uach] [ificarEncargado] c.carnet.uach.routes.NotificacionRoute : ? [ALERTA PARA EL ENCARGADO]
--- [uach] [ificarEncargado] c.carnet.uach.routes.NotificacionRoute : Mensaje: El estudiante con matrícula 200001 ha subido una evidencia para el evento ID: 2
--- [uach] [ificarEncargado] c.carnet.uach.routes.NotificacionRoute : =====
```

```
[uach] [ficarEstudiante] c.carnet.uach.routes.NotificacionRoute : ? [NOTIFICACIÓN ENVIADA AL ESTUDIANTE]
[uach] [ficarEstudiante] c.carnet.uach.routes.NotificacionRoute : Mensaje: Lo sentimos. Tu evidencia para el evento ID: 2 ha sido RECHAZADA.
[uach] [ficarEstudiante] c.carnet.uach.routes.NotificacionRoute : =====
[uach] [pool-3-thread-4] c.c.uach.routes.EvidenciaRechazadaRoute : Evidencia eliminada físicamente del disco de forma asíncrona: ..\uploads\evidencias\e8b6af69-8c86-4cee-91a0-
```

REDIS

La implementación de Redis es almacenar la sesión de un usuario en memoria para estar evitando y checando la base de datos, pero primero establecemos la conexión al servicio de redis, el puerto se queda en 6379.

```
# =====
# Configuración Redis
# =====
spring.data.redis.host=localhost
spring.data.redis.port=6379

# =====
# Configuración Caché y Sesiones (Opcional, pero explícito)
# =====
spring.cache.type=redis
spring.session.store-type=redis
```

Ahora si establecemos que se guarde en Redis la sesión de los usuarios.

```
package com.carnet.uach.config;

import org.springframework.context.annotation.Configuration;
import org.springframework.session.data.redis.config.annotation.web.http.EnableRedisHttpSession;

@Configuration
// Esta anotación intercepta todas las sesiones HTTP de los usuarios y,
// en lugar de guardarlas en la memoria temporal del servidor Tomcat,
// las guarda permanentemente en la base de datos en memoria Redis.
@EnableRedisHttpSession(maxInactiveIntervalInSeconds = 3600) // 1 hora de sesión
public class RedisConfig {
}
}
```

Cualquier acción que use la sesión de HTTP va ser redireccionado a lo que se guarda en redis gracias a Spring Boot, siempre se genera una sesión única, no sera posible ya que también se implementa un servicio de autenticador.

```
2) "\xac\xed\x00\x00\x00\x00myKey"  
127.0.0.1:6379> scan 0  
1) "0"  
2) 1) "pruba"  
   2) "spring:session:sessions:0a3ef4eb-76bb-4b3c-8333-d962ebd44727"  
   3) "\xac\xed\x00\x05t\x00\x05myKey"
```

Con todo esto cumplimos con los requisitos, el link del repositorio es:

<https://github.com/Magickyte/Proyecto-Final-Desarrollo-Plataformas>